



香港中文大學(深圳)

The Chinese University of Hong Kong, Shenzhen

Operating Systems Lecture 17

Memory: caching and paging

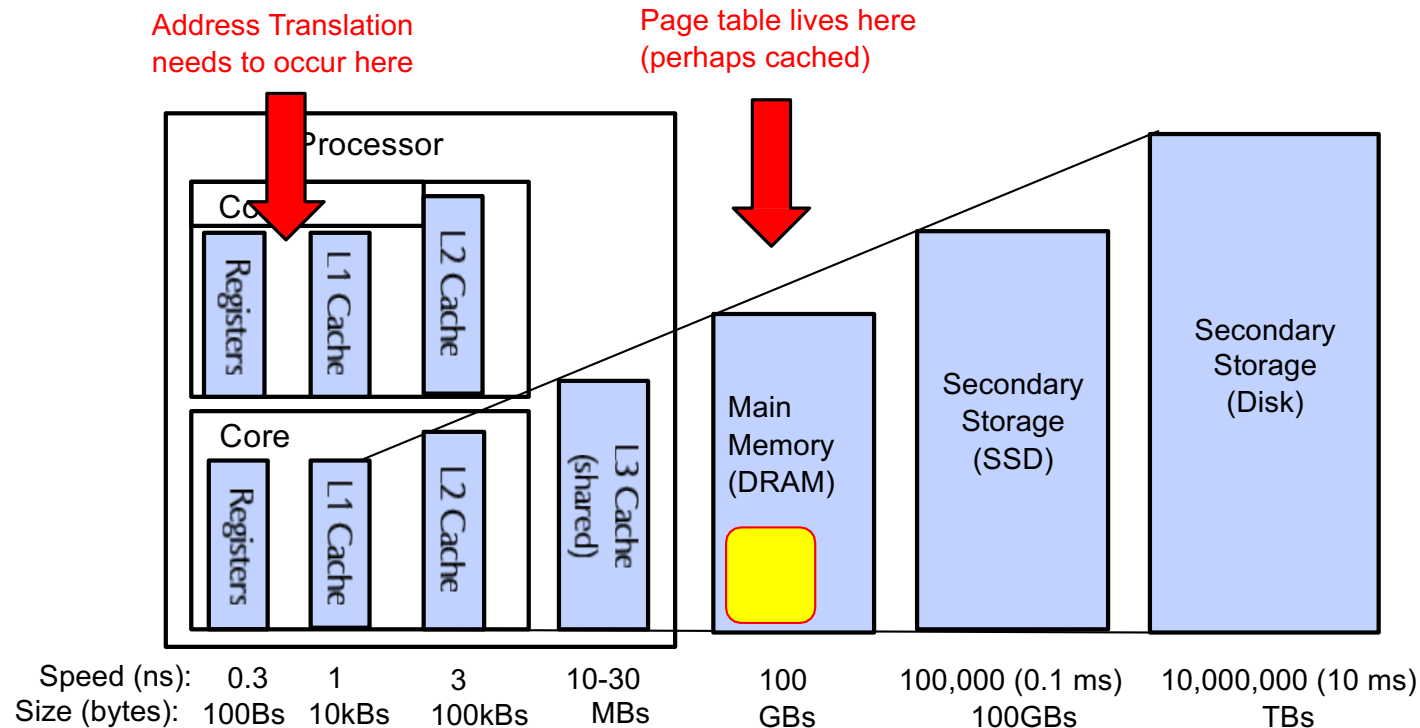
Prof. Yunming XIAO
School of Data Science

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Caching (continued)

Recall: memory hierarchy

- Caching: take advantage of the principle of locality to:
 - Present the illusion of having as much memory as in the cheapest technology
 - Provide average speed similar to that offered by the fastest technology



Recall: what happens on a write?

- **Write through:** The information is written to both the block in the cache and to the block in the lower-level memory
- **Write back:** The information is written only to the block in the cache
 - Modified cache block is written to main memory only when it is replaced
 - Question is block clean or dirty?

Recall: what happens on a write?

- Pros and Cons of each?
 - Write through:
 - PRO: read misses cannot result in writes
 - CON: Processor held up on writes unless writes buffered
 - Write back:
 - PRO:
 - repeated writes not sent to DRAM
 - processor not held up on writes
 - CON:
 - More complex
 - Read miss may require writeback of dirty data

Sources of cache misses

- **Compulsory** (cold start or process migration, first reference): first access to a block
 - “Cold” fact of life: not a whole lot you can do about it unless you prefetch
 - Solution: prefetch values before use
 - Note: If you run “billions” of instruction, Compulsory Misses are insignificant

Sources of cache misses

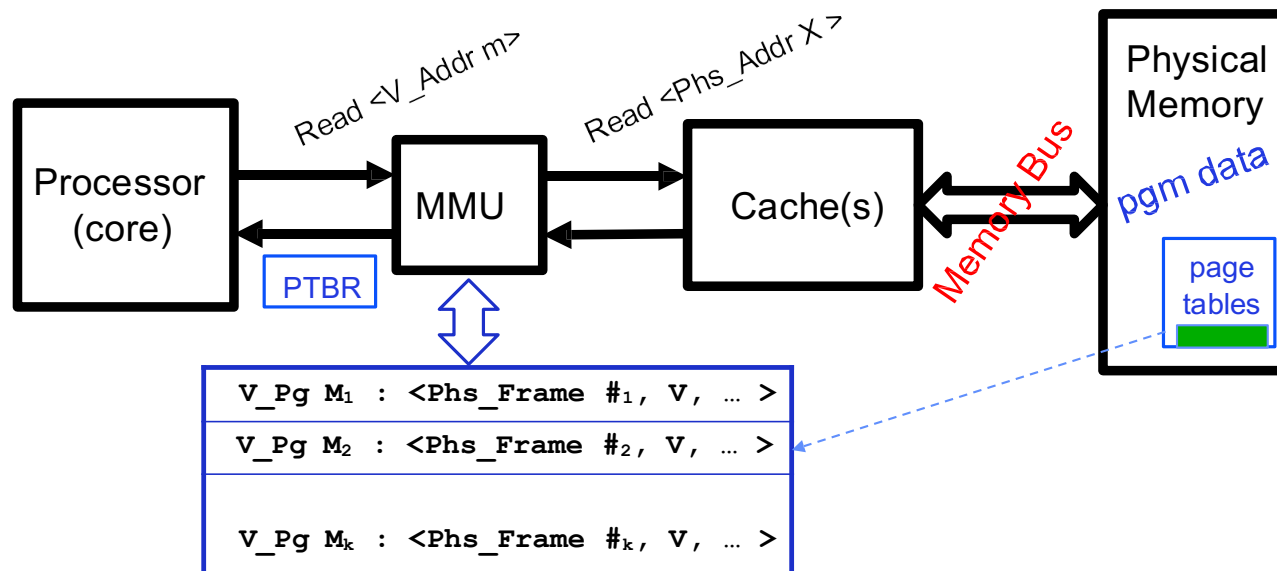
- **Capacity:**
 - Cache cannot contain all blocks access by the program
 - Solution 1: increase cache size
 - “Solution 2”: Change (e.g. reduce) associativity to focus misses in a few places?!
 - Consider fully-associative cache of size n : access pattern $0, 1, \dots, n-1, n, 0, 1, \dots$
 - Contrast with direct mapped of size n : Fewer misses!

Sources of cache misses

- **Conflict (collision):**
 - Multiple memory locations mapped to the same cache location
 - Solution 1: increase cache size
 - Solution 2: increase associativity
- **Coherence (Invalidation):**
 - other process (e.g., I/O) updates memory

How do we make Address Translation fast?

- Cache results of recent translations !
 - Different from a traditional cache
 - Cache Page Table Entries using Virtual Page # as the key



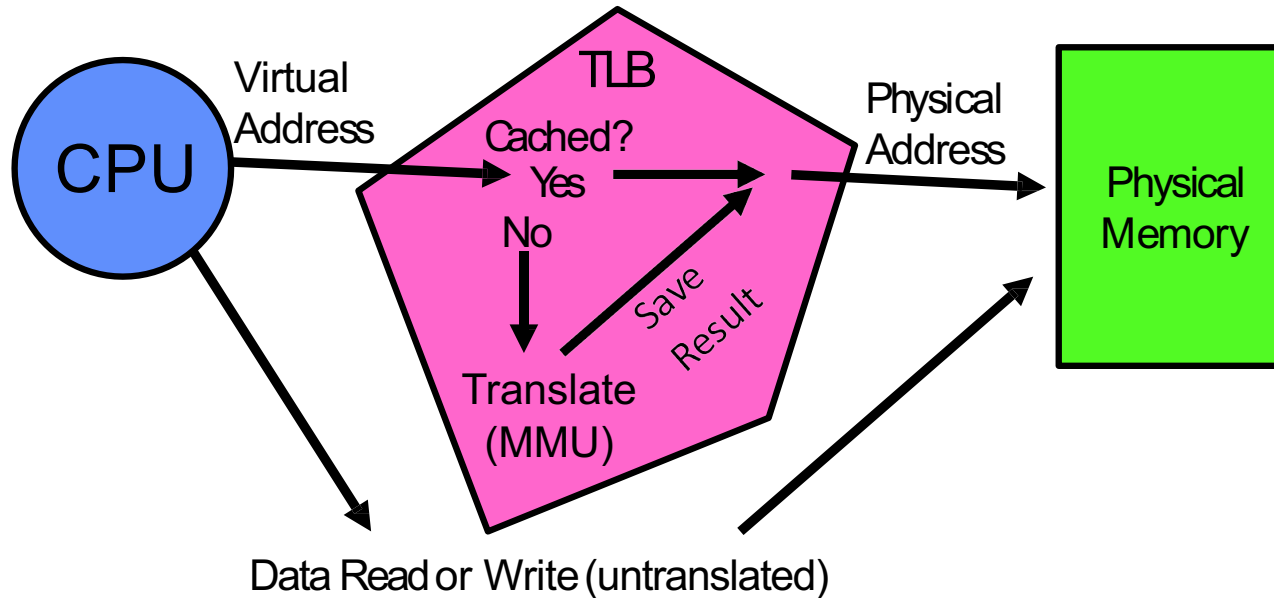
Translation Look-Aside Buffer (TLB)

- Record recent Virtual Page # to Physical Frame # translation
- If present, have the physical address without reading any page tables !!!
 - Even if the translation involved multiple levels
 - Caches the end-to-end result
- Was invented by Sir Maurice Wilkes – *prior to caches*
 - When you come up with a new concept, you get to name it!
 - People realized “if it’s good for page tables, why not the rest of the data in memory?”
- On a *TLB miss*, the page tables may be cached, so only go to memory when both miss



Maurice Wilkes
Turing Award: 1967
(1913 – 2010)

Caching applied to address translation

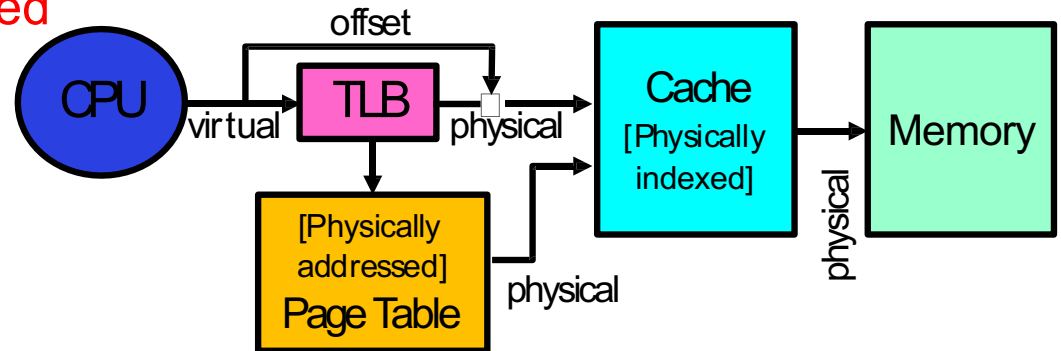


- Question is about page locality: does it exist?
 - Instruction accesses spend a lot of time on same page (accesses are sequential)
 - Stack accesses have definite locality of reference
 - Data accesses have less page locality, but still some...
- Can we have a TLB hierarchy?
 - Sure: multiple levels at different sizes/speeds

Physically-Indexed vs Virtually-Indexed caches

- **Physically-Indexed, Physically-Tagged**

- Address handed to cache after translation
- Page Table in physical memory (so that it can be cached)
- Benefits:
 - Every piece of data has single place in cache
 - Cache can stay unchanged on context switch
- Challenges:
 - TLB is in critical path of lookup!
- Pretty common today (e.g. x86 processors)



Physically-Indexed vs Virtually-Indexed caches

- **Virtually-Indexed, Virtually-Tagged or Physically-Tagged**

- Address handed to cache before translation
- Page Table in virtual memory (so that it can be cached); Only last level of Page Table points to physical memory.

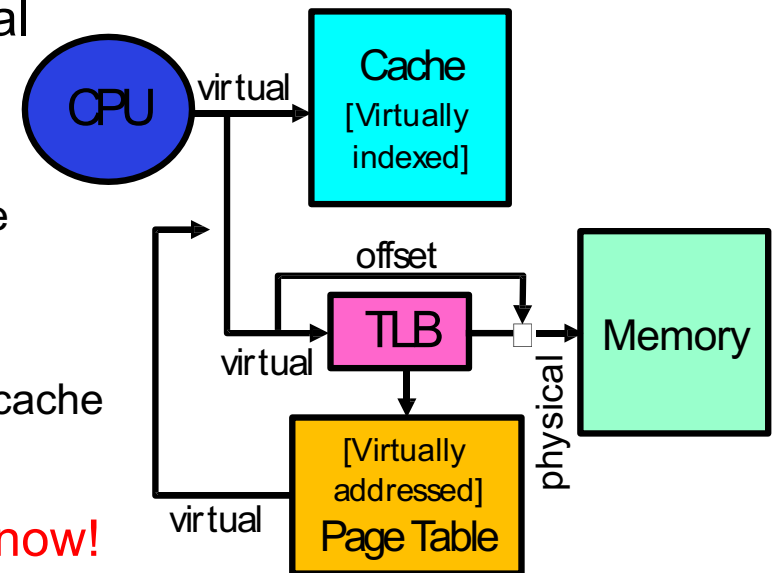
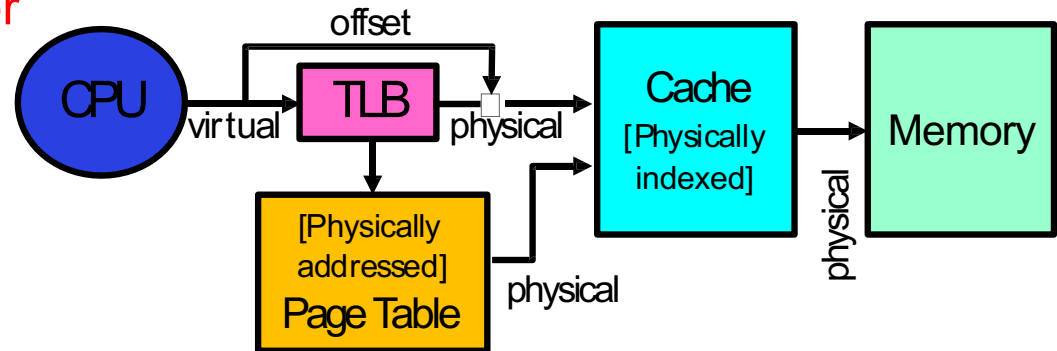
- **Benefits:**

- TLB not in critical path of lookup, so system can be faster

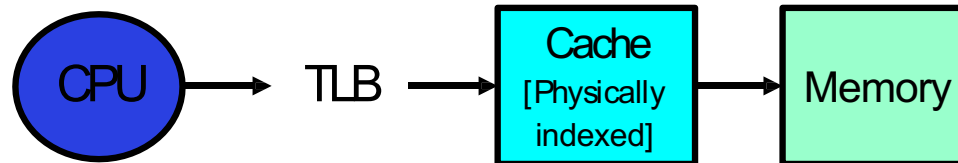
- **Challenges:**

- Same data could be mapped in multiple places of cache
- May need to flush cache on context switch

- **We will stick with Physically Indexed Caches for now!**

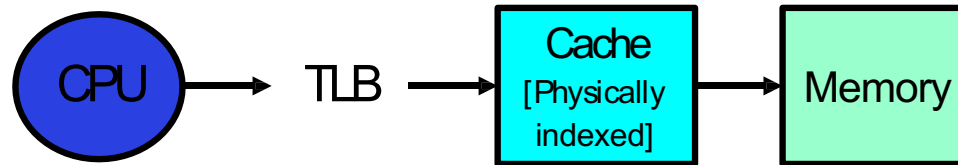


What TLB organization makes sense?



- For Physically Indexed/Tagged, needs to be really fast
 - Critical path of memory access
 - In simplest view: before the cache
 - Thus, this adds to access time (reducing cache speed)
 - Seems to argue for Direct Mapped or Low Associativity
- However, needs to have very few conflicts!
 - With TLB, the Miss Time is extremely high! (Page Table traversal)
 - Cost of conflict (Miss Time) is high
 - Hit Time – dictated by clock cycle

What TLB organization makes sense?

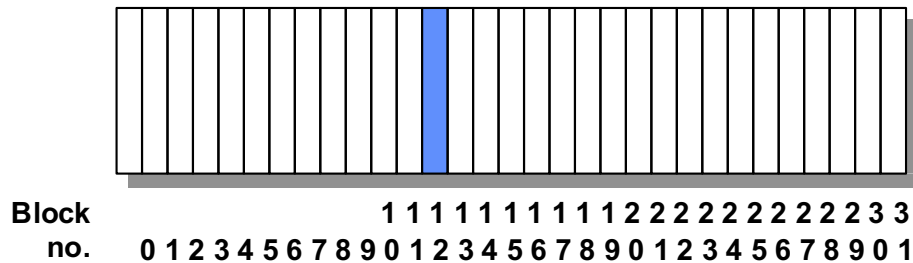


- **Thrashing:** continuous conflicts between accesses
 - What if use low order bits of virtual page number as index into TLB?
 - First page of code, data, stack may map to same entry
 - Need 3-way associativity at least?
 - What if use high order bits as index?
 - TLB mostly unused for small programs

Recall: where does a Block get placed in a Cache?

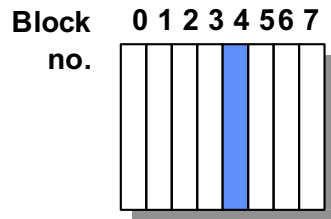
- Example: Block 12 placed in 8 block cache

32-Block Address Space:



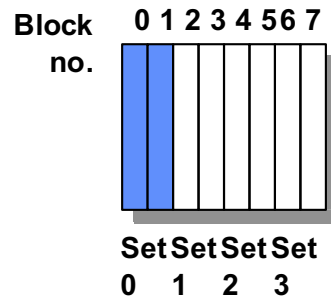
Direct mapped:

**block 12 can go
only into block 4
(12 mod 8)**



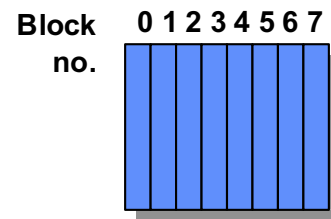
Set associative:

**block 12 can go
anywhere in set 0
(12 mod 4)**



Fully associative:

**block 12 can go
anywhere**



TLB organization: include protection

- How big does TLB actually have to be?
 - Usually small: 128-512 entries (larger now)
 - Not very big, can support higher associativity
- Small TLBs usually organized as fully-associative cache
 - Lookup is by Virtual Address
 - Returns Physical Address + other info
- What happens when fully-associative is too slow?
 - Put a small (4-16 entry) direct-mapped cache in front
 - Called a “TLB Slice”

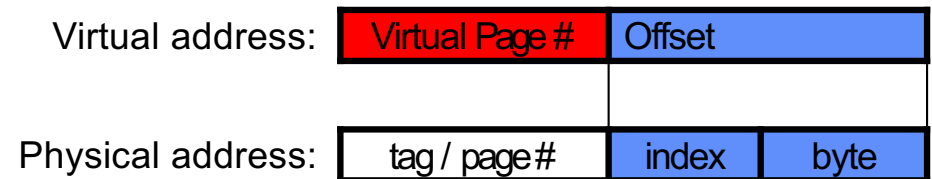
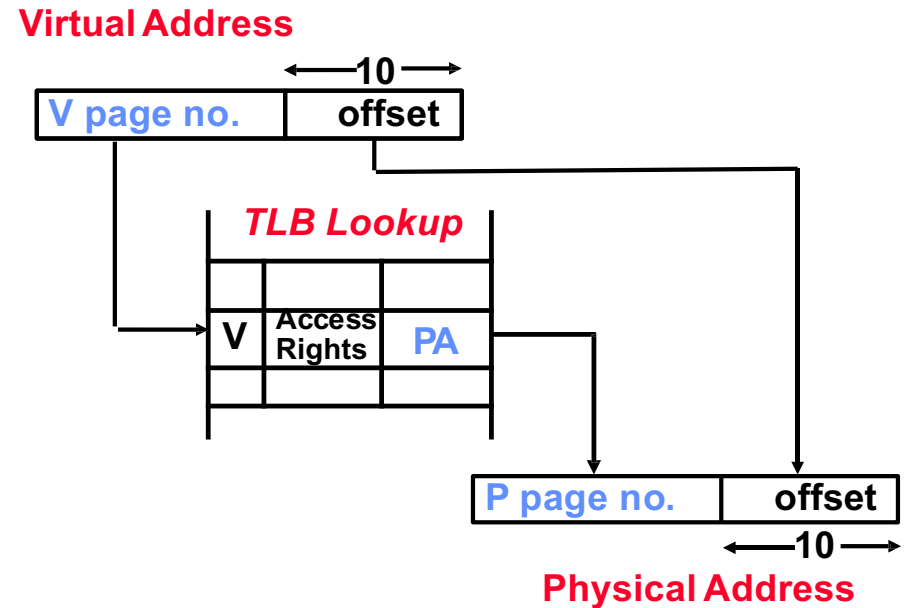
TLB organization: include protection

- Example for MIPS R3000:

Virtual Address	Physical Address	Dirty	Ref	Valid	Access	ASID
0xFA00	0x0003	Y	N	Y	R/W	34
0x0040	0x0010	N	Y	Y	R	0
0x0041	0x0011	N	Y	Y	R	0

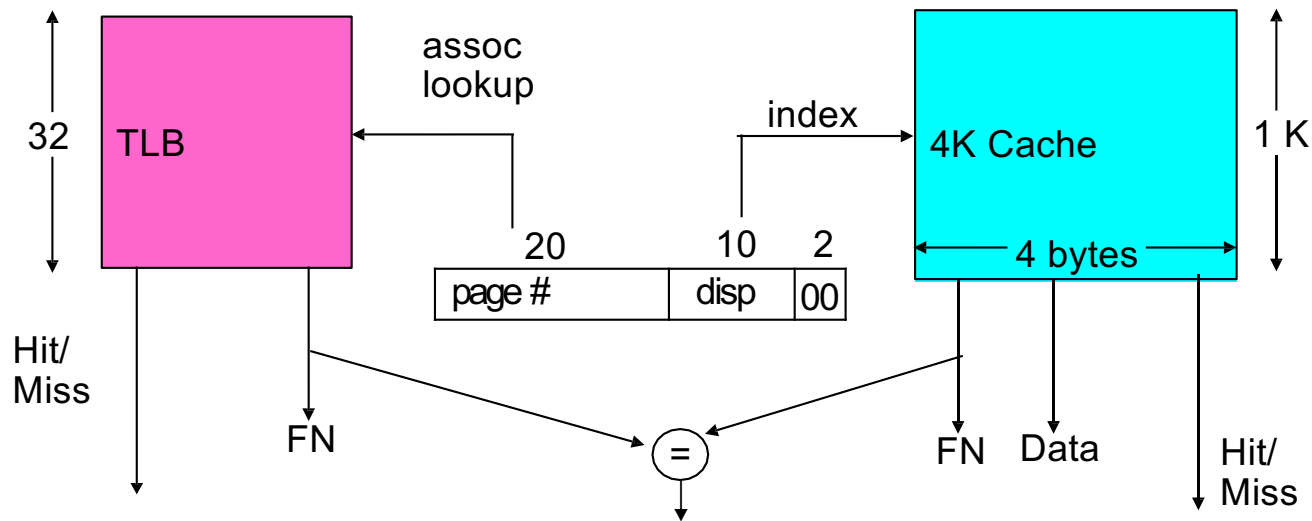
Further reducing translation time for physically-indexed caches

- As described, TLB lookup is in serial with cache lookup
 - Consequently, speed of TLB can impact speed of access to cache
- Machines with TLBs go one step further: overlap TLB lookup with cache access
 - Works because offset available early
 - Offset in virtual address exactly covers the “cache index” and “byte select”
 - Thus, can select the cached byte(s) in parallel to perform address translation



Overlapping Cache and TLB access

- Here is how this might work with a 4K cache:



- What if cache size is increased to 8KB?
 - Overlap not complete
 - Need to do something else

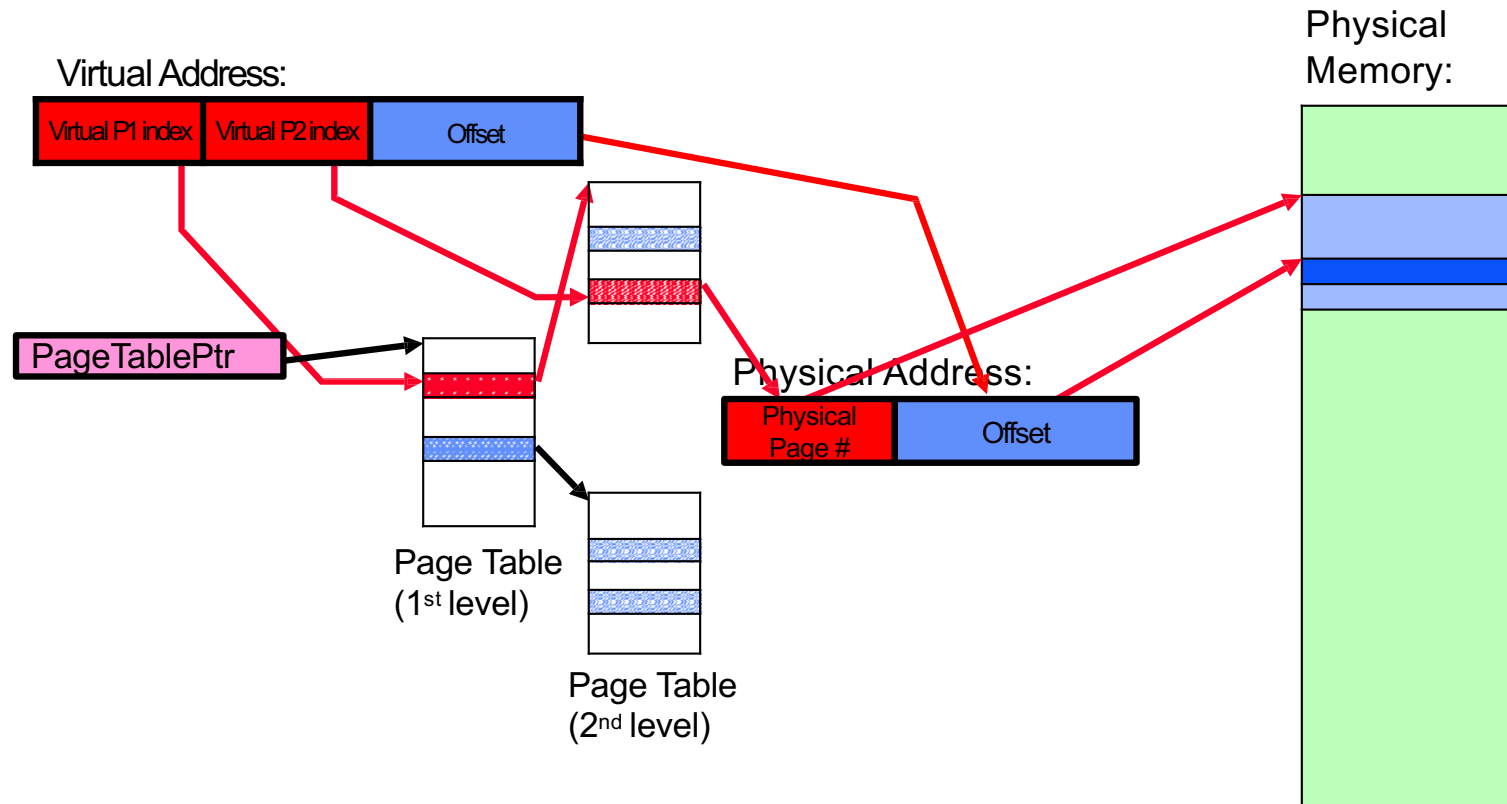
Summary

- Major categories of cache misses:
 - **Compulsory Misses:** sad facts of life. Example: cold start misses.
 - **Conflict Misses:** increase cache size and/or associativity
 - **Capacity Misses:** increase cache size
 - **Coherence Misses:** Caused by external processors or I/O devices
- Cache organizations:
 - Direct Mapped: single block per set
 - Set associative: more than one block per set
 - Fully associative: all entries equivalent

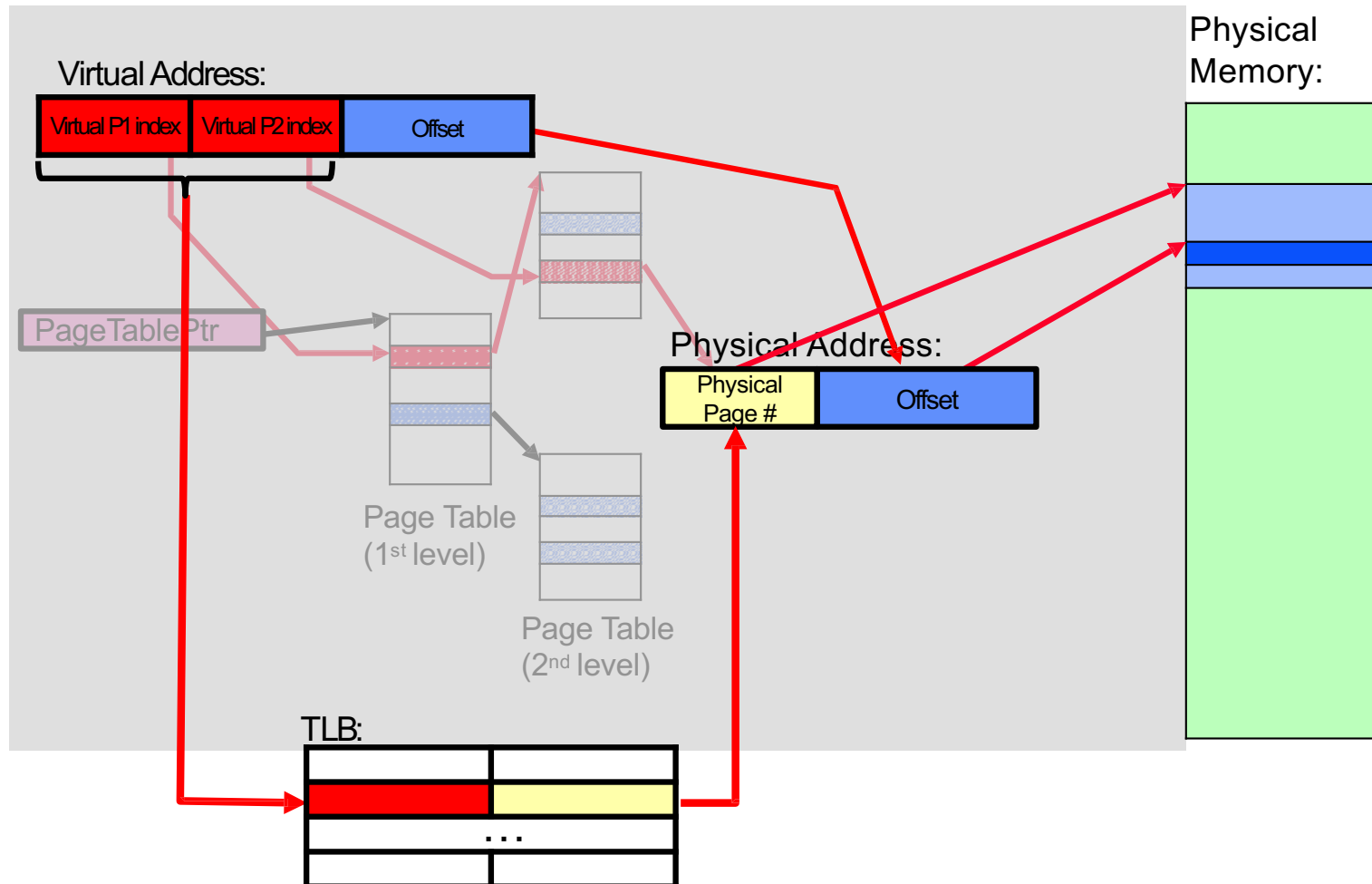
Summary

- “Translation Lookaside Buffer” (TLB)
 - Small number of PTEs and optional process IDs (< 512)
 - Often Fully Associative (Since conflict misses expensive)
 - On TLB miss, page table must be traversed and if located PTE is invalid, cause Page Fault
 - On change in page table, TLB entries must be invalidated

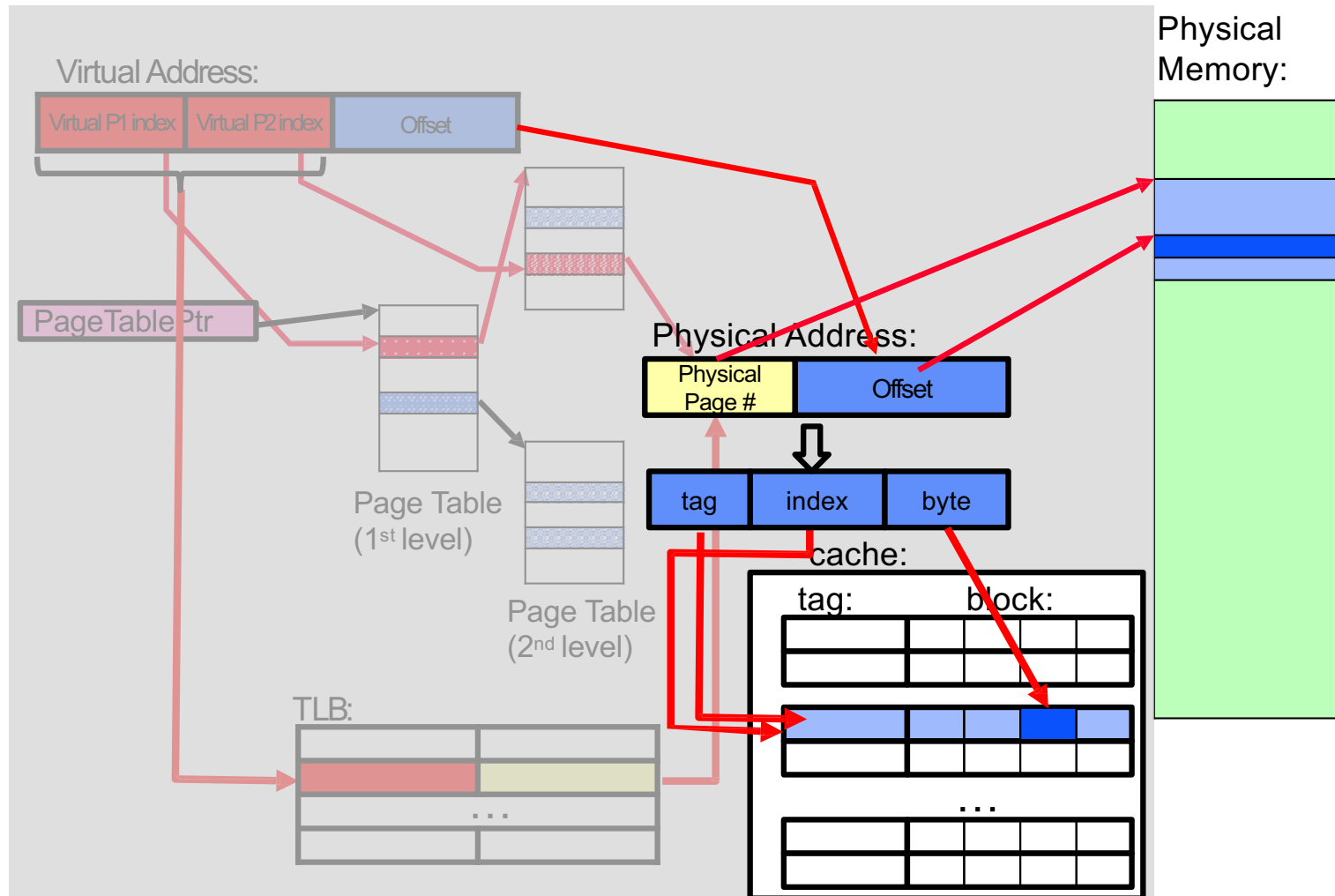
Putting everything together: address translation



Putting everything together: TLB



Putting everything together: cache



Demand paging

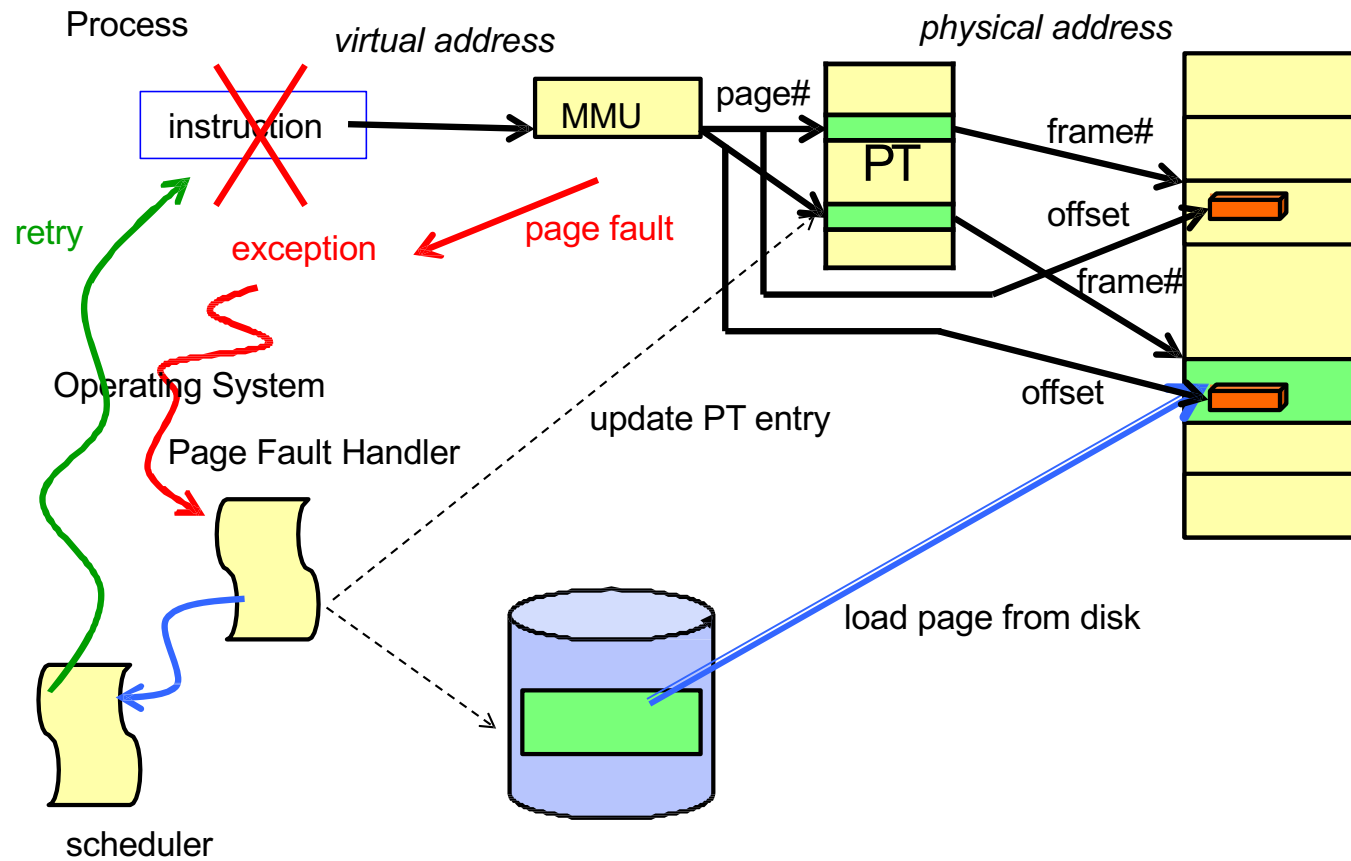
Page fault handling

- The Virtual-to-Physical Translation fails
 - PTE marked invalid, Privilege Level Violation, Access violation, or does not exist
 - Causes a Fault / Trap
 - Not an interrupt because synchronous to instruction execution
 - May occur on instruction fetch or data access
 - Protection violations typically terminate the process

Page fault handling

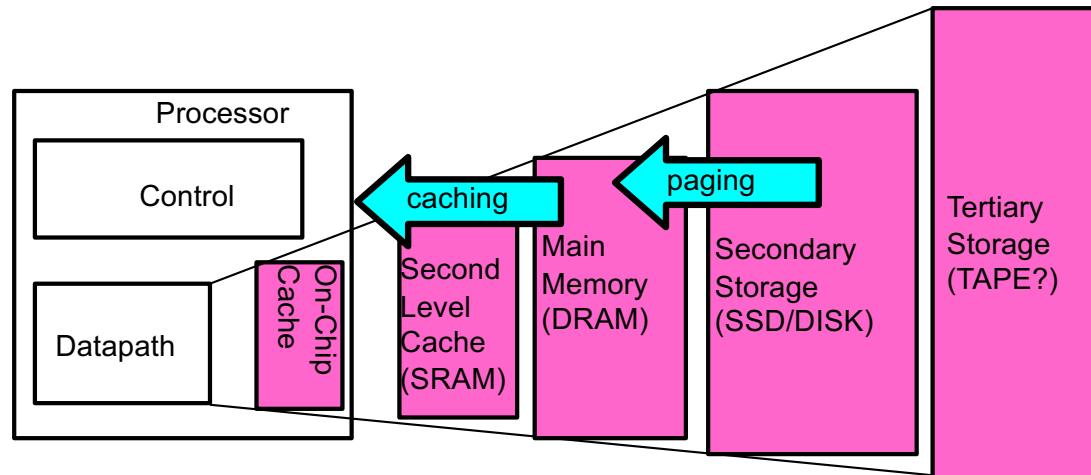
- Other Page Faults engage operating system to fix the situation and retry the instruction
 - Allocate an additional stack page, or
 - Make the page accessible – (Copy on Write),
 - Bring page in from secondary storage to memory – demand paging
- Fundamental inversion of the hardware / software boundary
 - Need to execute software to allow hardware to proceed!

Page Fault $\Rightarrow$ Demand Paging

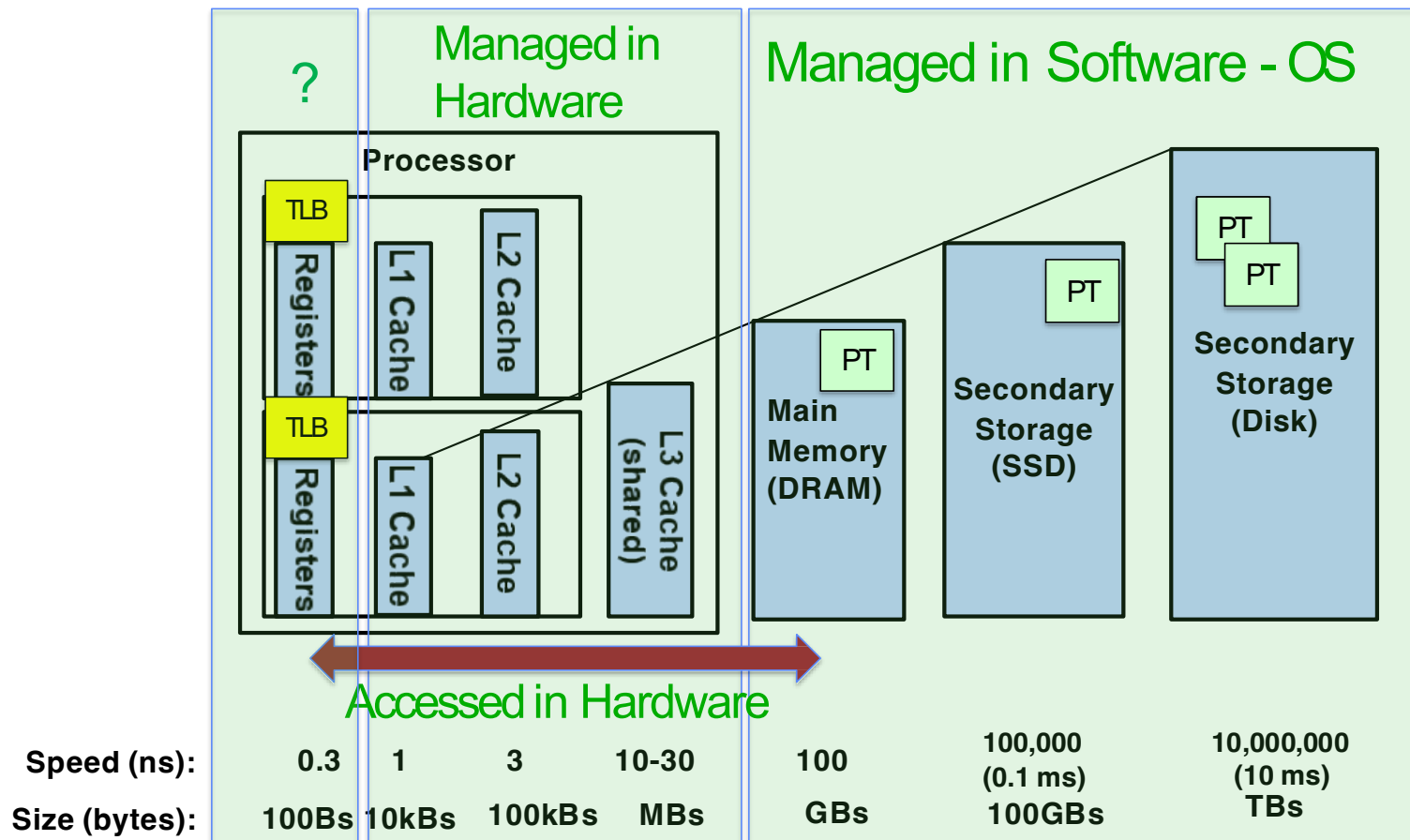


Demand paging

- Modern programs require a lot of physical memory
 - Memory per system growing faster than 25%-30%/year
- But they don't use all their memory all of the time
 - 90-10 rule: programs spend 90% of their time in 10% of their code
 - Wasteful to require all of user's code to be in memory
- Solution: use main memory as “cache” for disk



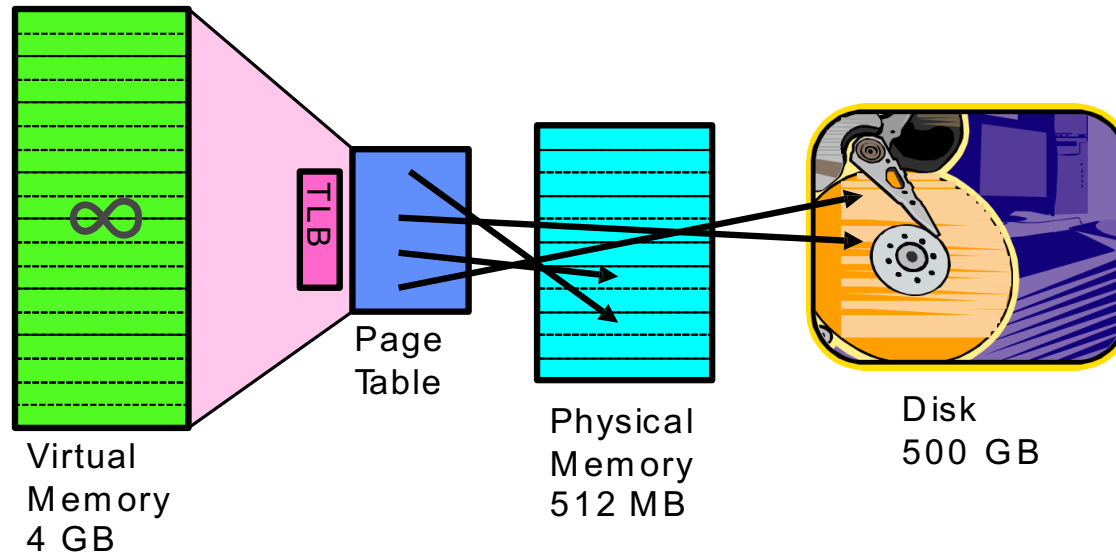
Management & Access to the memory hierarchy



Demand paging as caching, ...

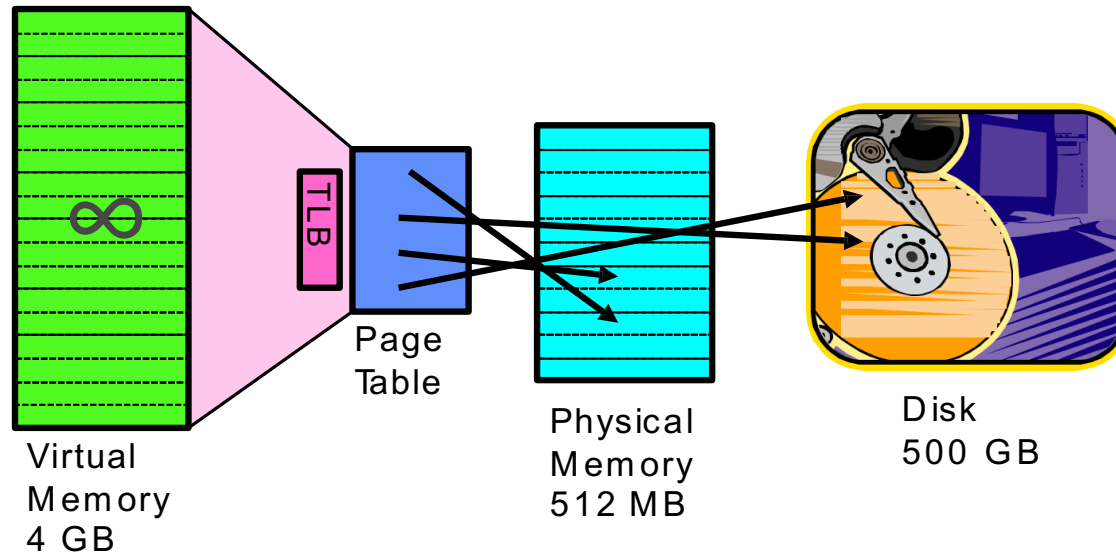
- What “block size”? - 1 page (e.g, 4 KB)
- What “organization” ie. direct-mapped, set-assoc., fully-associative?
 - Fully associative since arbitrary virtual → physical mapping
- How do we locate a page?
 - First check TLB, then page-table traversal
- What is page replacement policy? (i.e. LRU, Random...)
 - This requires more explanation... (next lecture)
- What happens on a miss?
 - Go to lower level to fill miss (i.e. disk)
- What happens on a write? (write-through/write back?)
 - Definitely write-back – need dirty bit!

Illusion of infinite memory



- Disk is larger than physical memory $\Rightarrow$
 - In-use virtual memory can be bigger than physical memory
 - Combined memory of running processes much larger than physical memory
 - More programs fit into memory, allowing more concurrency

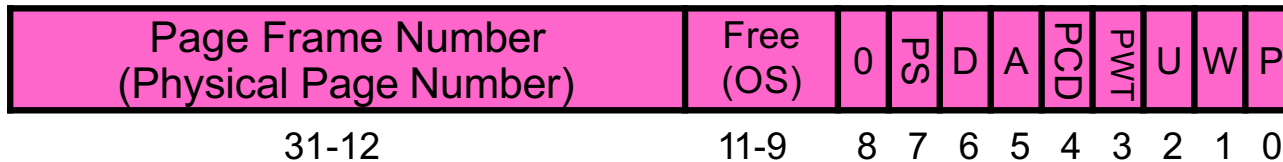
Illusion of Infinite Memory



- Principle: **Transparent Level of Indirection** (page table)
 - Supports flexible placement of physical data
 - Data could be on disk or somewhere across network
 - Variable location of data transparent to user program
 - Performance issue, not correctness issue

Recall: what is in a Page Table Entry (PTE)?

- What is in a Page Table Entry (or PTE)?
 - Pointer to next-level page table or to actual page
 - Permission bits: valid, read-only, read-write, write-only
- Example: Intel x86 architecture PTE:
 - 2-level page table (10, 10, 12-bit offset)
 - Intermediate page tables called “Directories”



- P:** Present (same as “valid” bit in other architectures)
- W:** Writeable
- U:** User accessible
- PWT:** Page write transparent: external cache write-through
- PCD:** Page cache disabled (page cannot be cached)
- A:** Accessed: page has been accessed recently
- D:** Dirty (PTE only): page has been modified recently
- PS:** Page Size: PS=1⇒4MB page (directory only).
Bottom 22 bits of virtual address serve as offset

Demand paging mechanisms

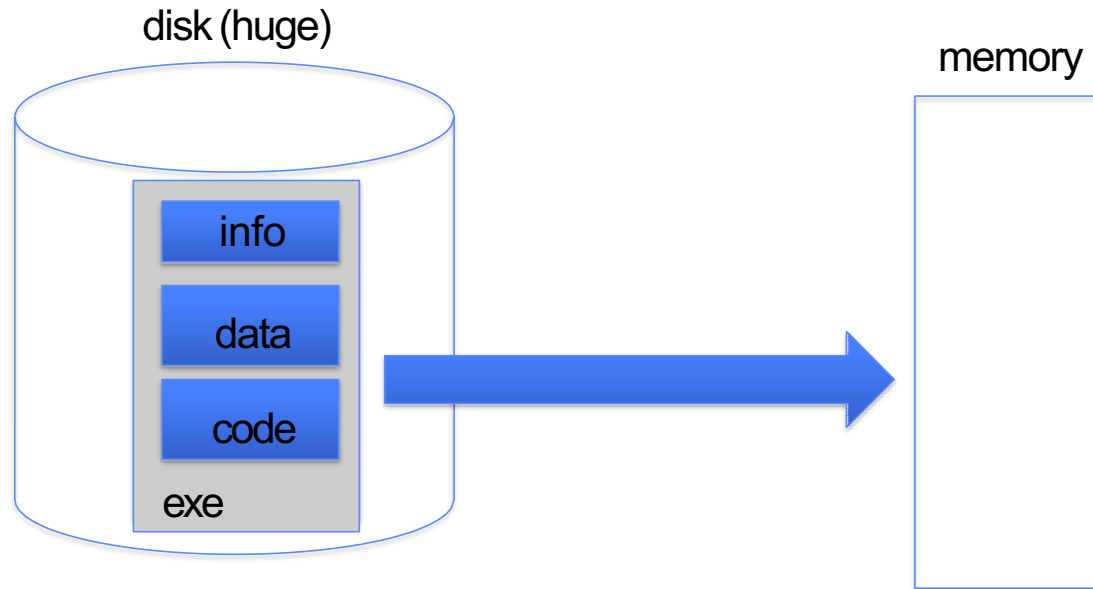
- PTE makes demand paging implementable
 - Valid $\Rightarrow$ Page in memory, PTE points at physical page
 - Not Valid $\Rightarrow$ Page not in memory; use info in PTE to find it on disk when necessary
- Suppose user references page with invalid PTE?
 - Memory Management Unit (MMU) traps to OS
 - Resulting trap is a “Page Fault”
 - What does OS do on a Page Fault?:
 - Choose an old page to replace
 - If old page modified (“D=1”), write contents back to disk
 - Change its PTE and any cached TLB to be invalid
 - Load new page into memory from disk
 - Update page table entry, invalidate TLB for new entry
 - Continue thread from original faulting location
 - TLB for new page will be loaded when thread continued!
 - While pulling pages off disk for one process, OS runs another process from ready queue
 - Suspended process sits on wait queue

cache

Many uses of virtual memory and “demand paging” ...

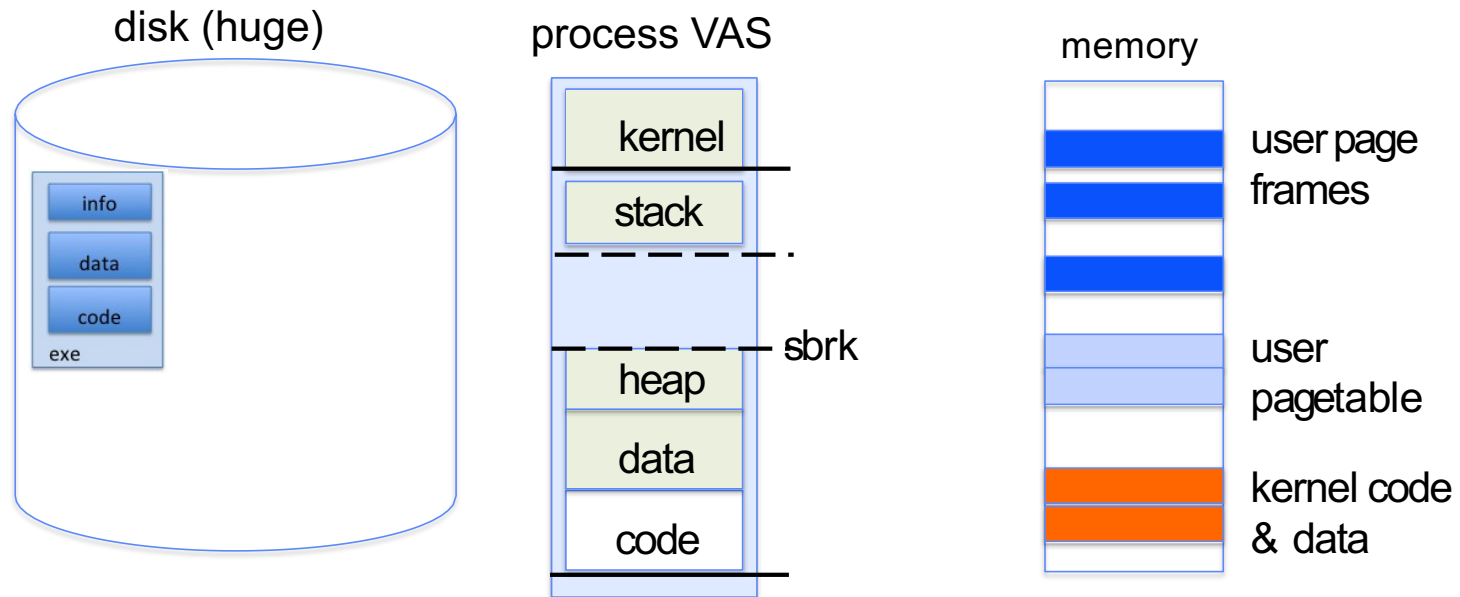
- Extend the stack
 - Allocate a page and zero it
- Extend the heap (sbrk of old, today mmap)
- Process Fork
 - Create a copy of the page table
 - Entries refer to parent pages – NO-WRITE
 - Shared read-only pages remain shared
 - Copy page on write
- Exec
 - Only bring in parts of the binary in active use
 - Do this on demand
- MMAP to explicitly share region (or to access a file as RAM)

Classic: loading an executable into memory



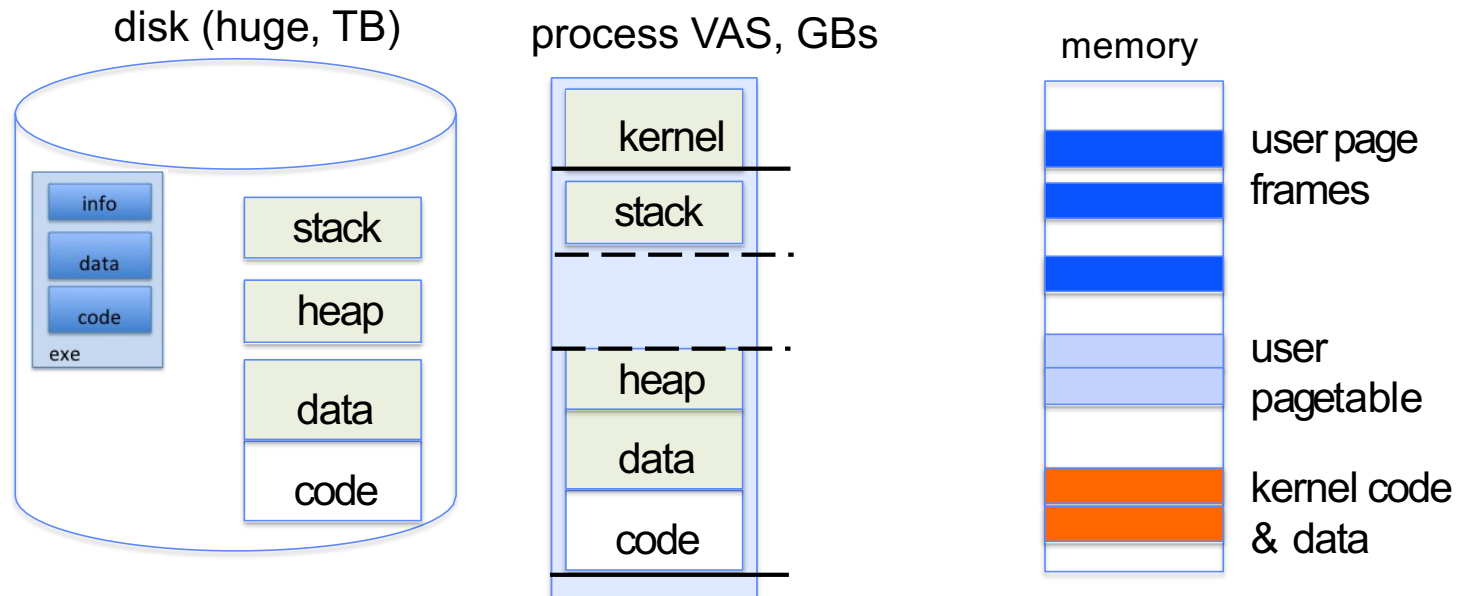
- .exe
 - lives on disk in the file system
 - contains contents of code & data segments, relocation entries and symbols
 - OS loads it into memory, initializes registers (and initial stack pointer)
 - program sets up stack and heap upon initialization:
 - `crt0` (C runtime init)

Create virtual address space of the process



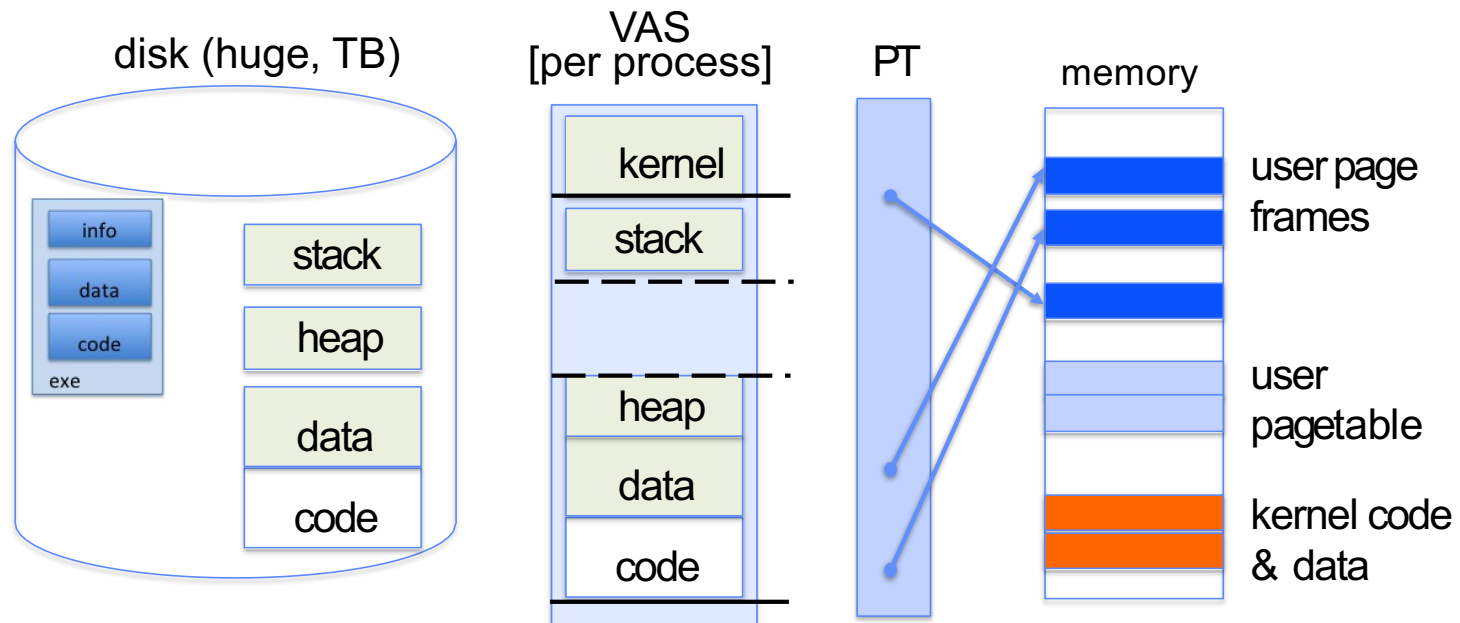
- Utilized pages in the virtual address space (VAS) are backed by a page block on disk
 - Called the backing store or swap file
 - Typically, in an optimized block store, but can think of it like a file

Create virtual address space of the process



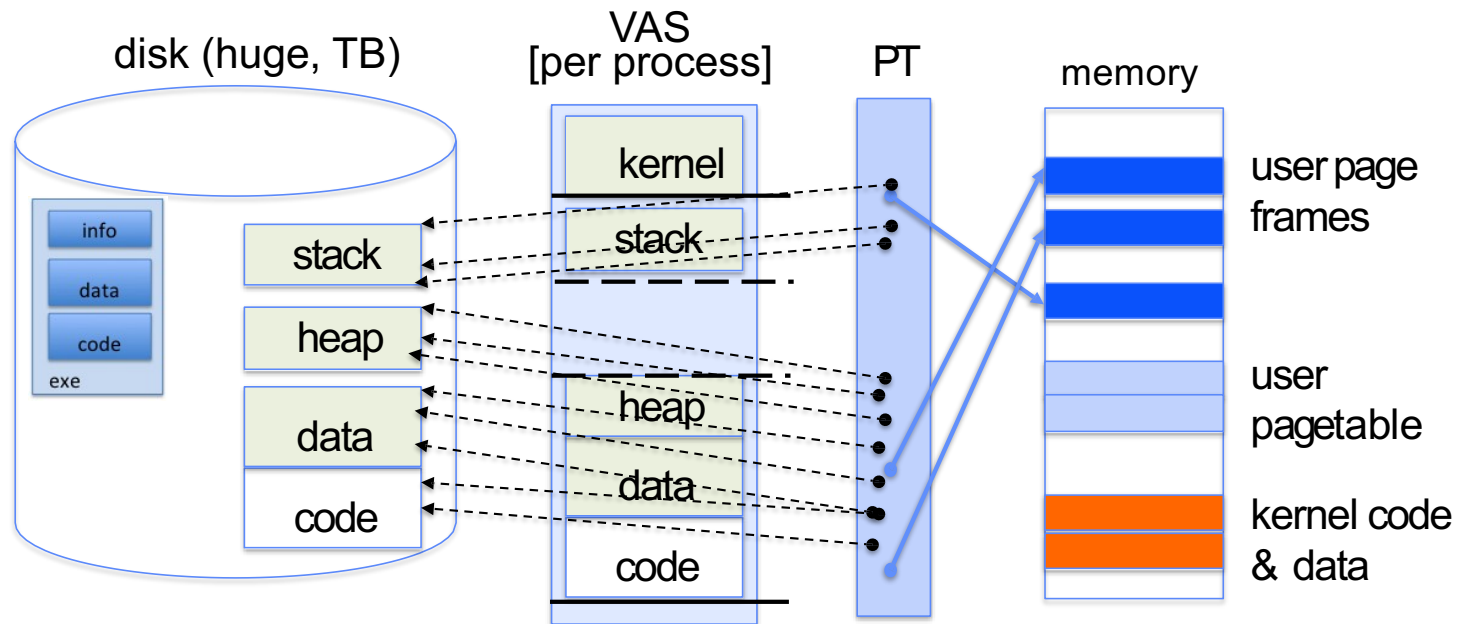
- User page table maps entire VAS
- All the utilized regions are backed on disk
 - swapped into and out of memory as needed
- For *every* process

Create virtual address space of the process



- User page table maps entire VAS
 - Resident pages to the frame in memory they occupy
 - The portion of it that the HW needs to access must be resident in memory

Provide backing store for VAS

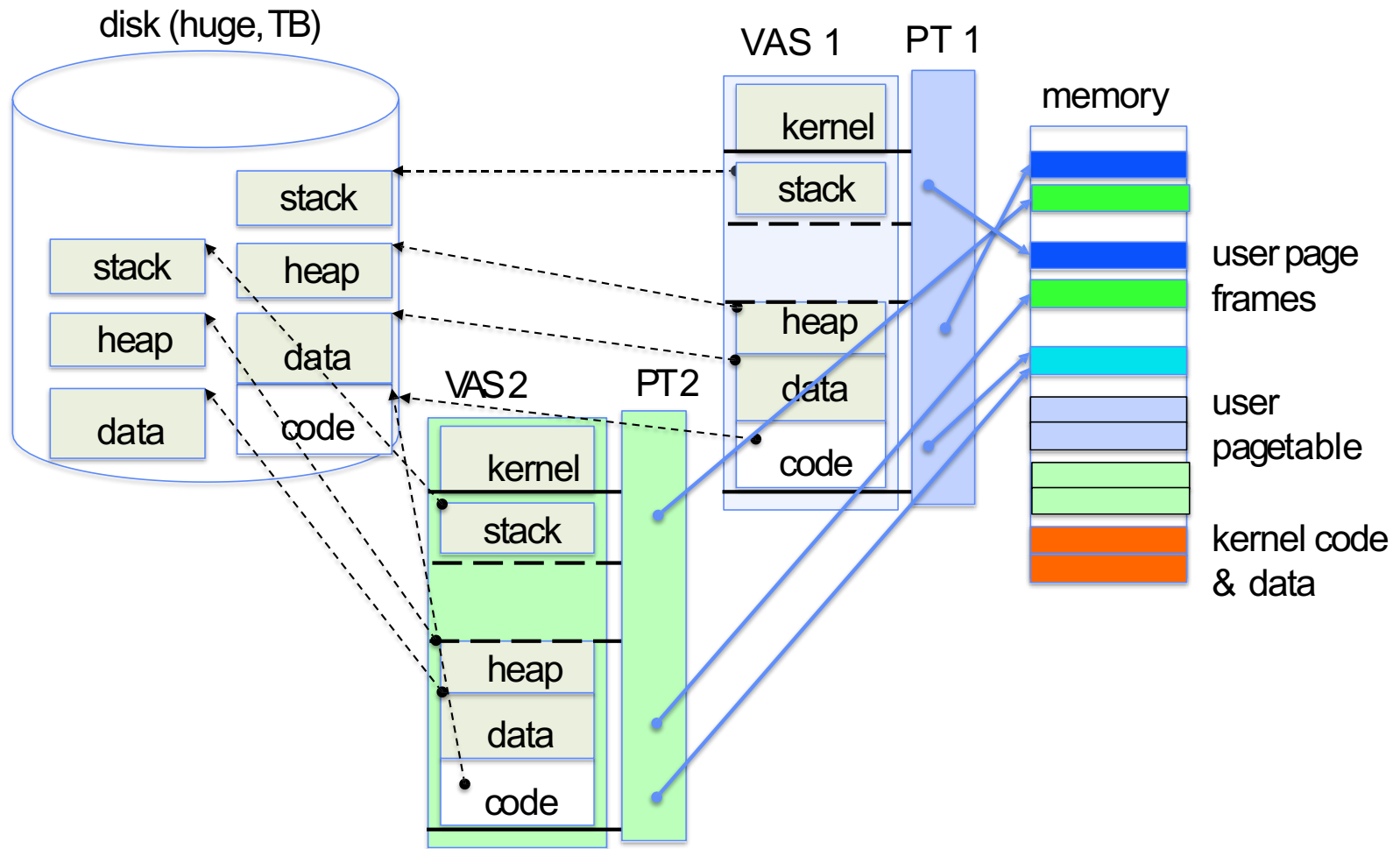


- User page table maps entire VAS
- Resident pages mapped to memory frames
- For all other pages, OS must record where to find them on disk
 - Many ways to do this, but might use remaining bits of PTE when P=0

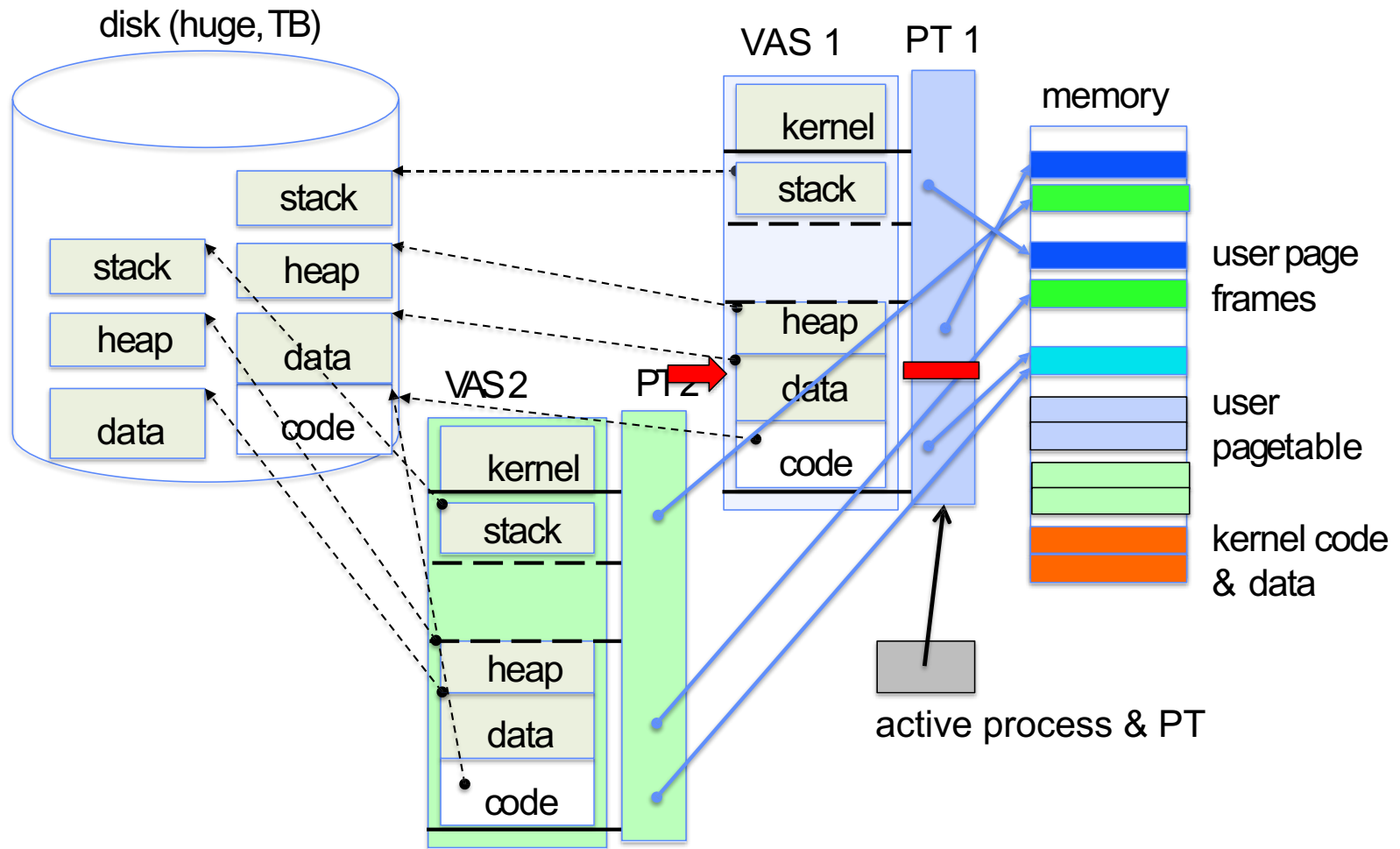
What data structure maps non-resident pages to disk?

- FindBlock(PID, page#) → disk_block
 - Some OSs utilize spare space in PTE for paged blocks
 - Like the PT, but purely software
- Where to store it?
 - In memory – can be compact representation if swap storage is contiguous on disk
 - Could use hash table (like Inverted PT)
- Usually want backing store for resident pages too
- May map code segment directly to on-disk image
 - Saves a copy of code to swap file
- May share code segment with multiple instances of the program

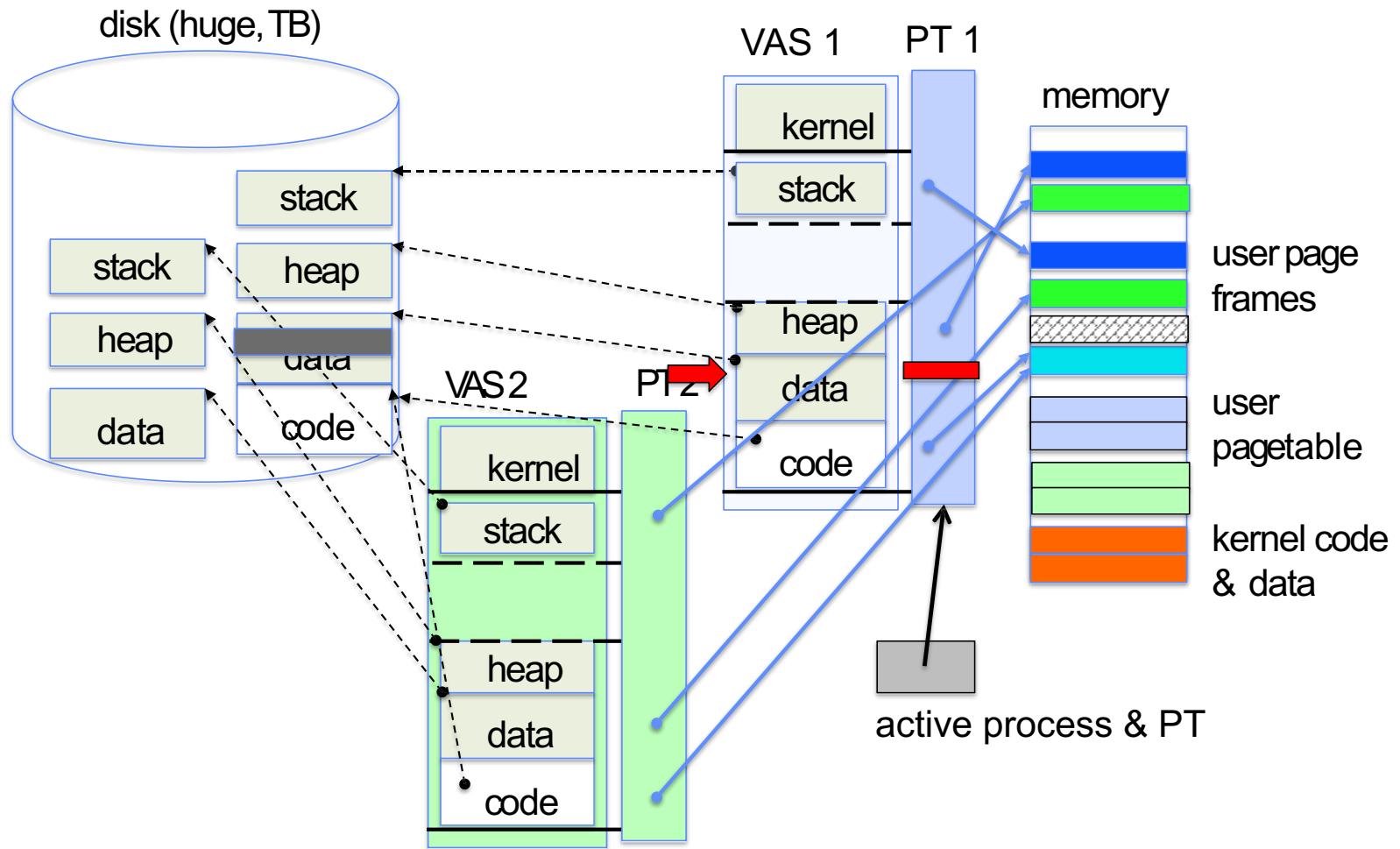
Provide backing store for VAS



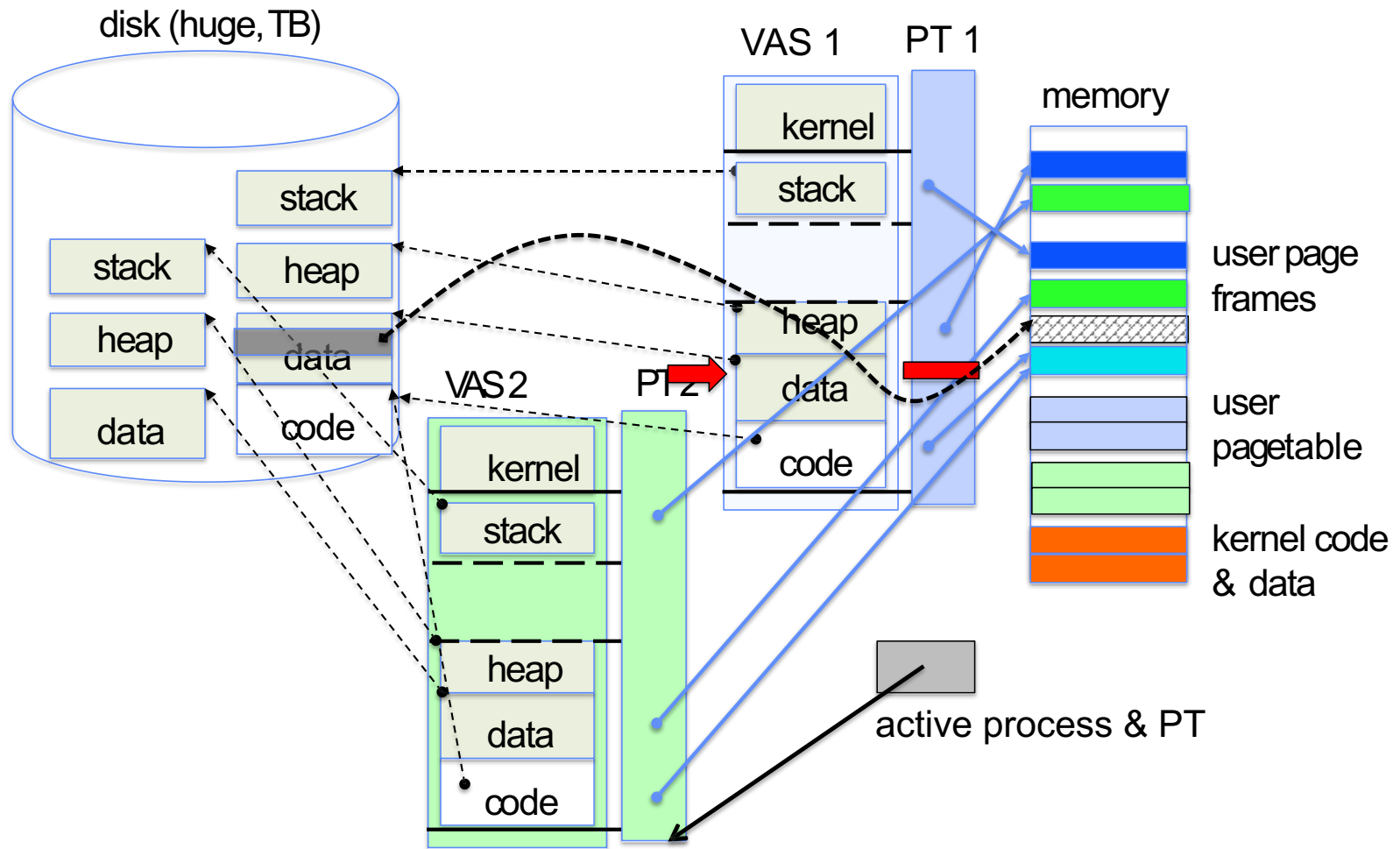
On page fault ...



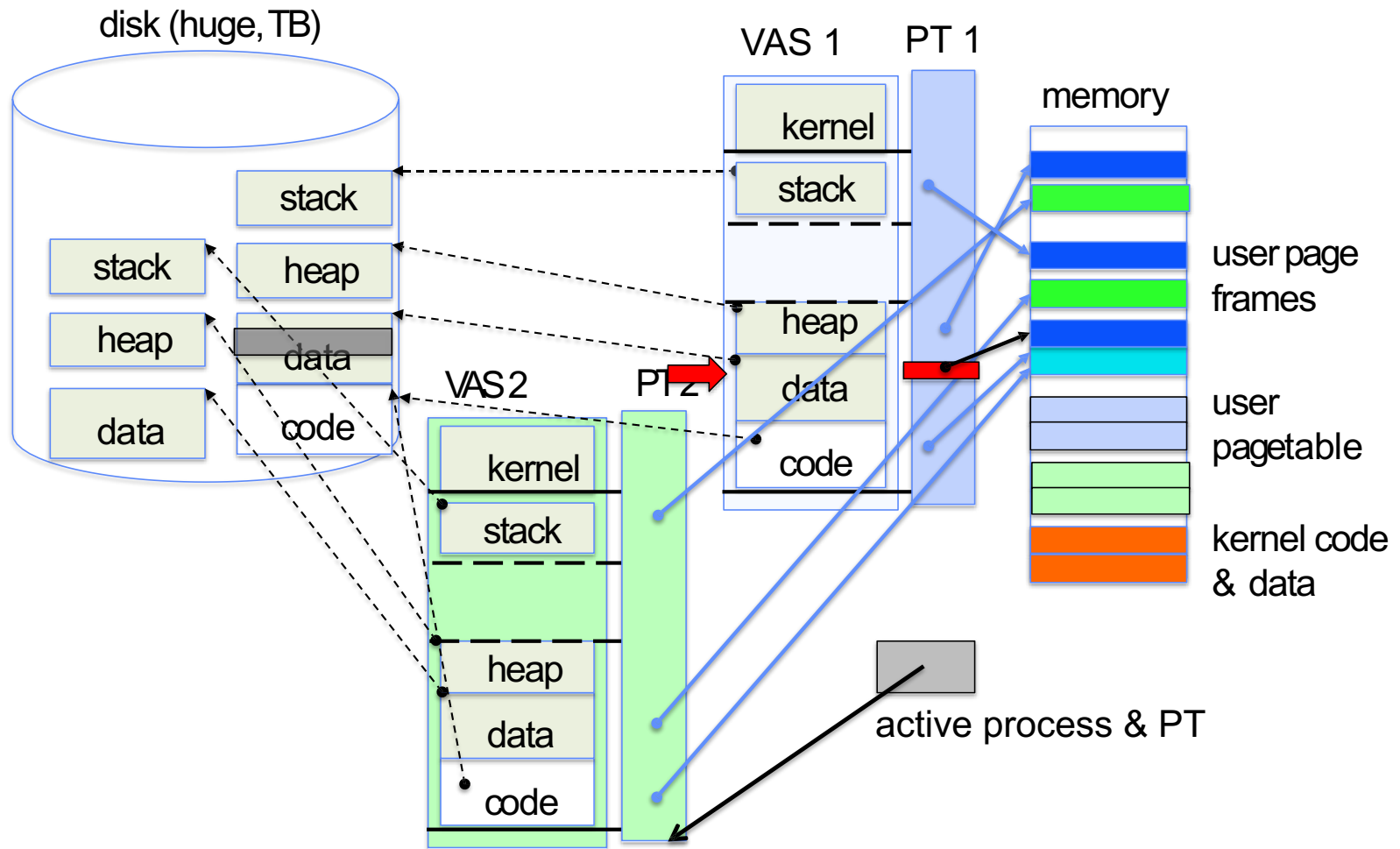
On page fault ... find & start load



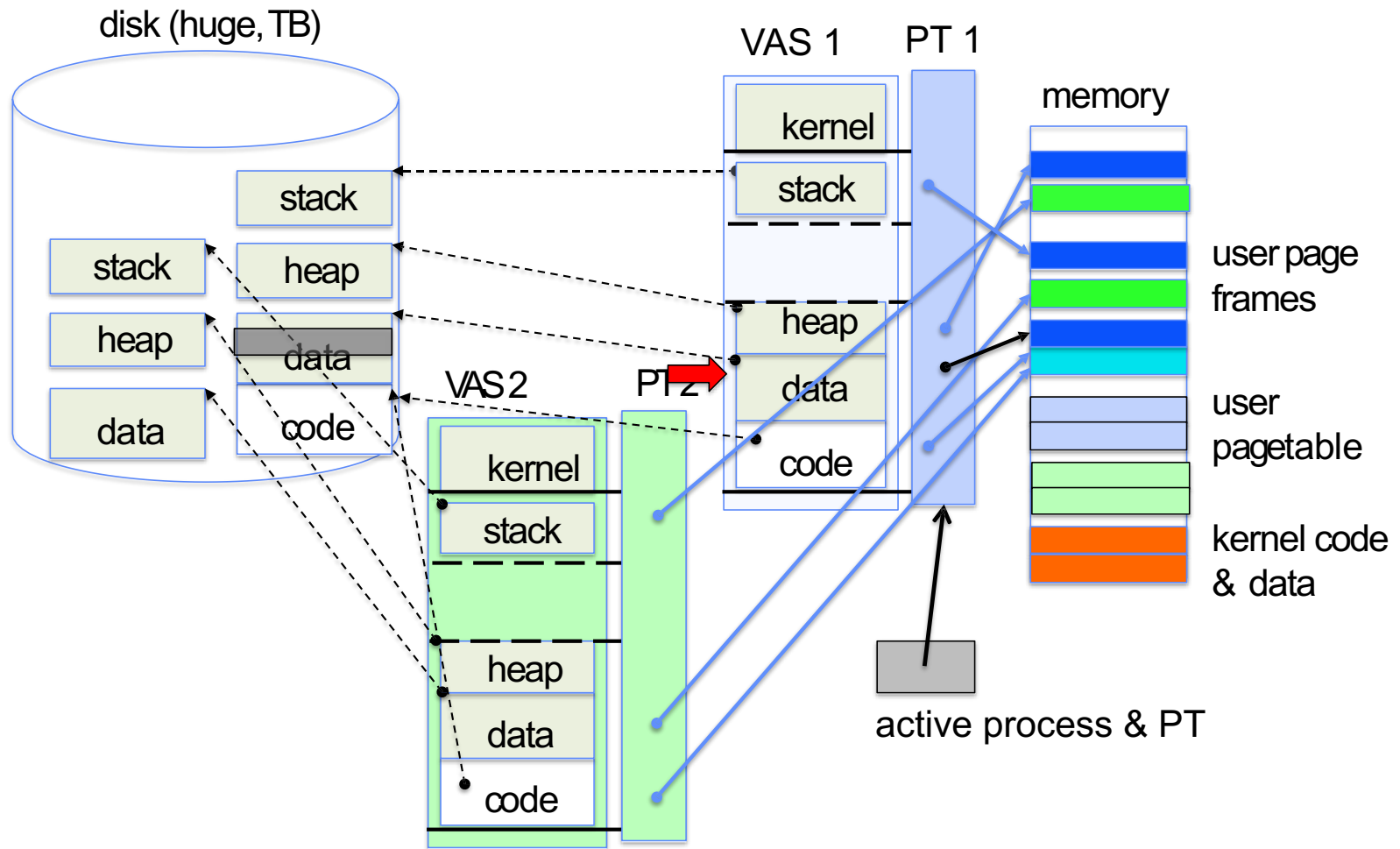
On page fault ... schedule other P or T



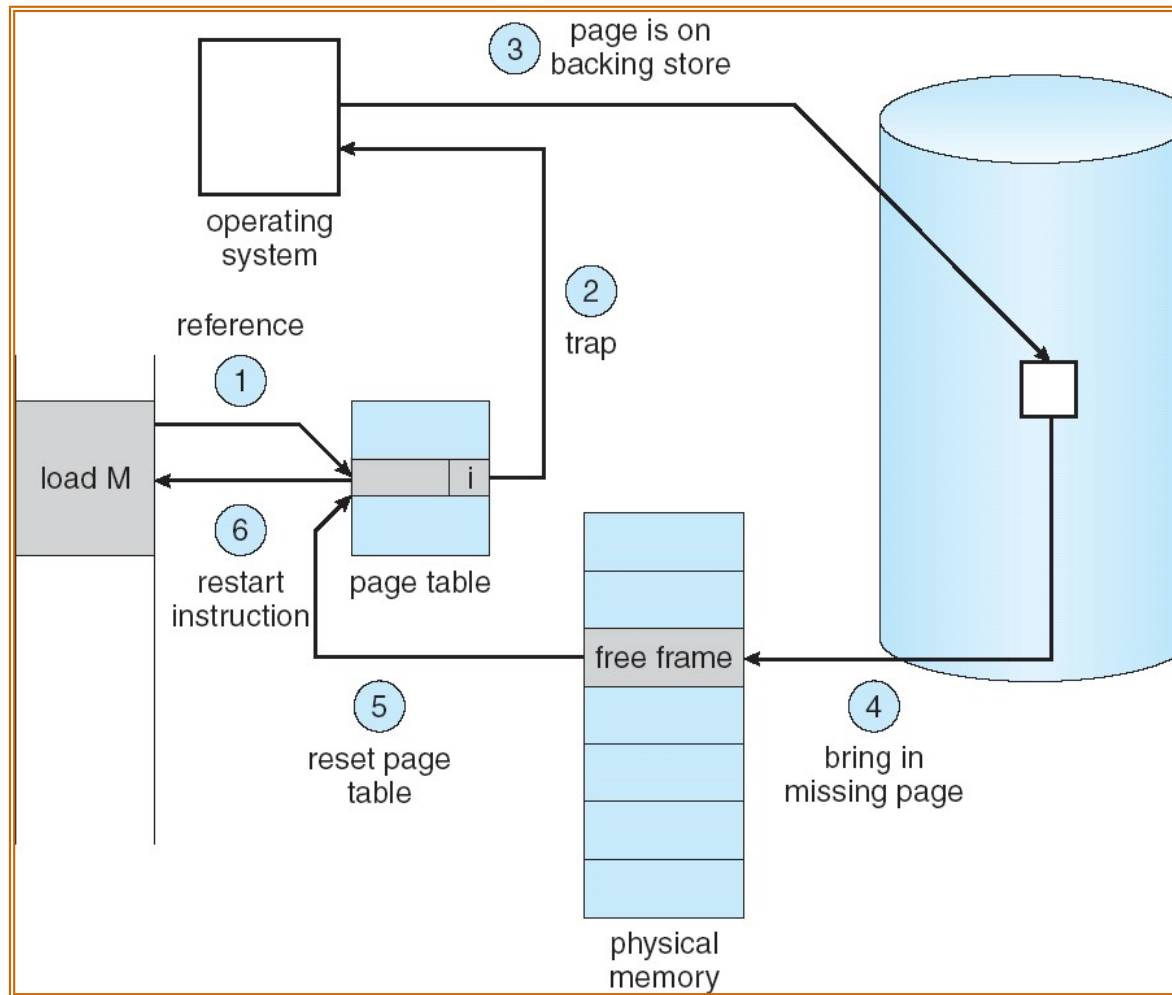
On page fault ... update PTE



Eventually reschedule faulting thread



Summary: steps in handling a page fault

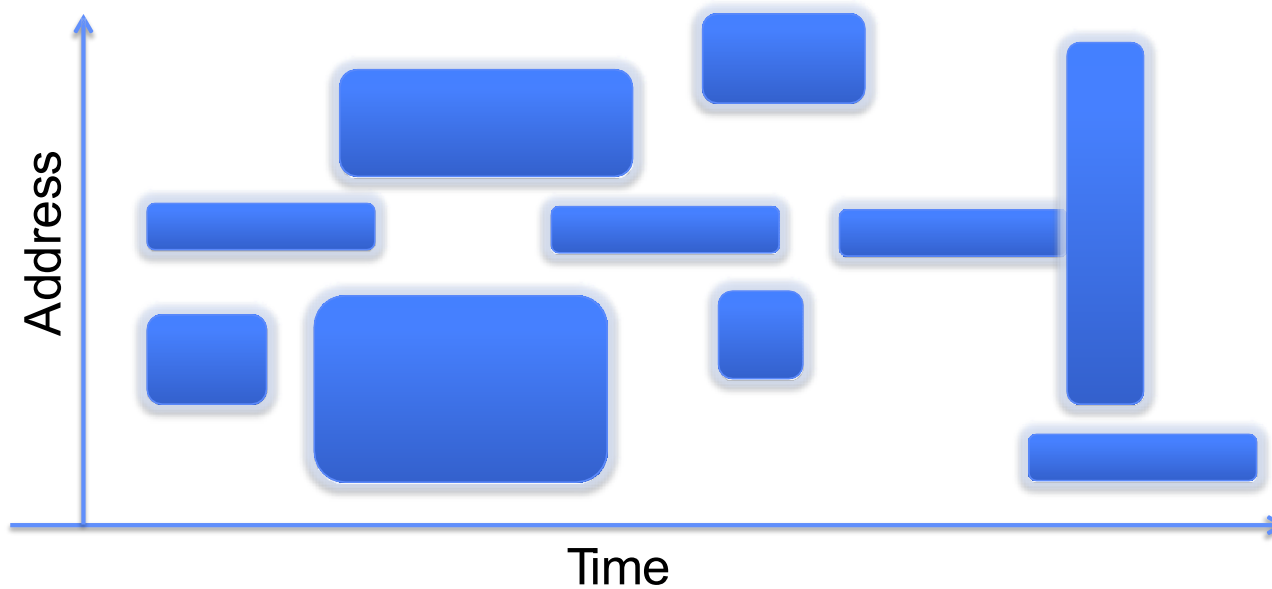


Some questions we need to answer!

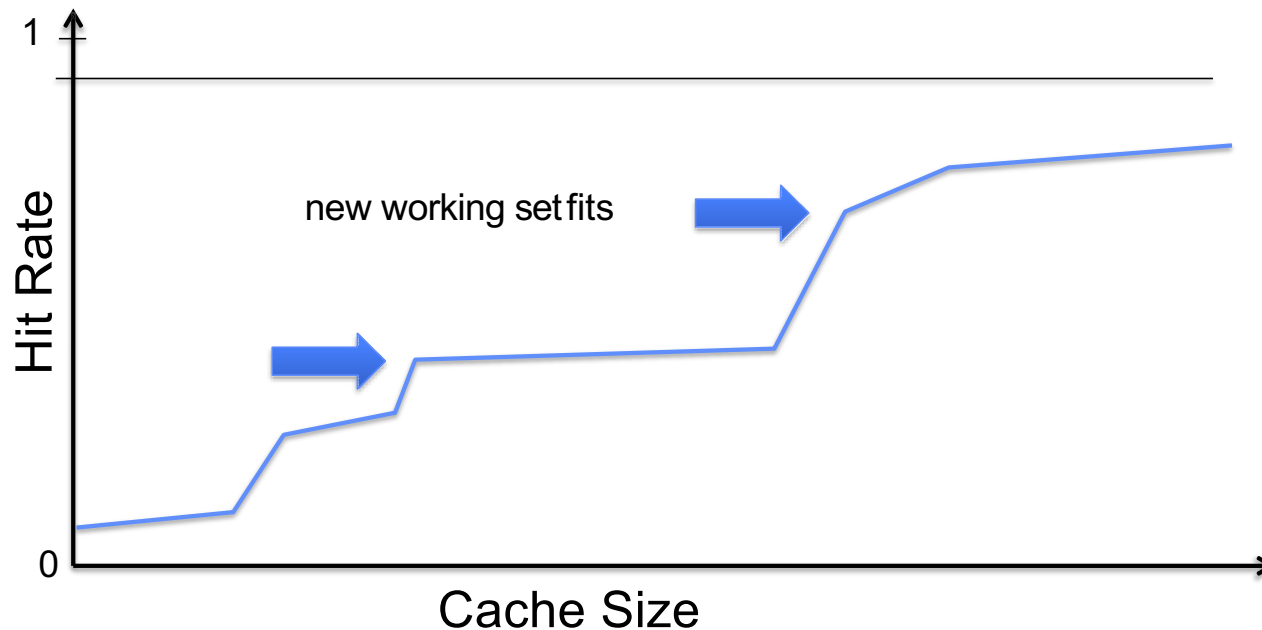
- During a page fault, where does the OS get a free frame?
 - Keeps a free list
 - Unix runs a “reaper” if memory gets too full
 - Schedule dirty pages to be written back on disk
 - Zero (clean) pages which haven’t been accessed in a while
 - As a last resort, evict a dirty page first
- How can we organize these mechanisms?
 - Work on the replacement policy
- How many page frames/process?
 - Like thread scheduling, need to “schedule” memory resources:
 - utilization? fairness? priority?
 - Allocation of disk paging bandwidth

Working set model

- As a program executes, it transitions through a sequence of “working sets” consisting of varying sized subsets of the address space



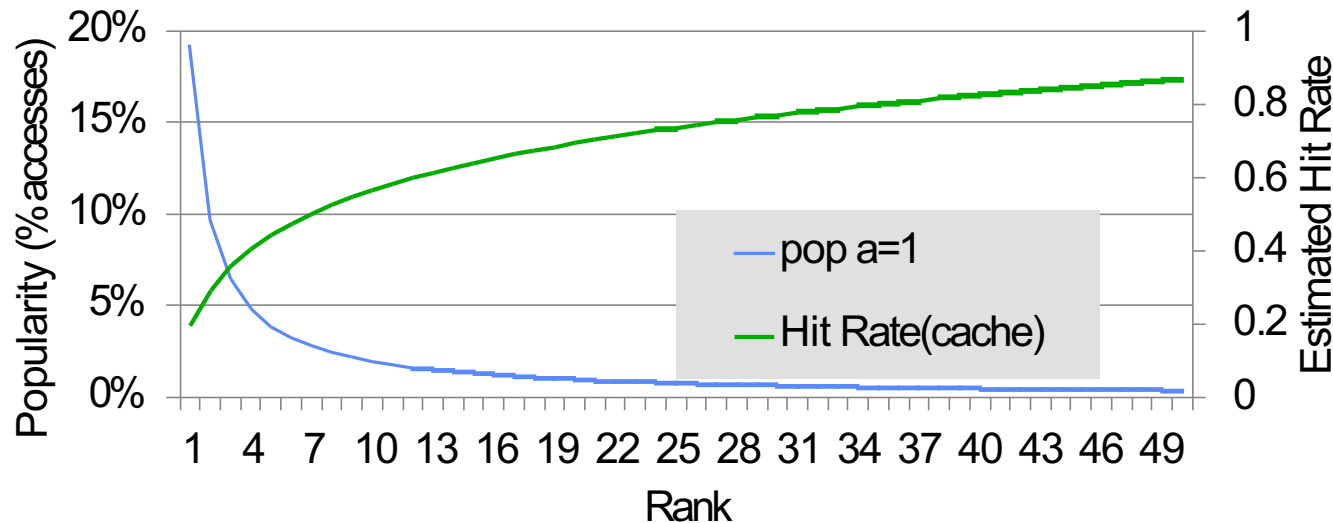
Cache behavior under WS model



- Amortized by fraction of time the Working Set is active
- Transitions from one WS to the next
- Capacity, Conflict, Compulsory misses
- Applicable to memory caches and pages. Others ?

Another model of locality: Zipf

$$P_{\text{access}}(\text{rank}) = 1/\text{rank}$$



- Likelihood of accessing item of rank r is $\propto 1/r^a$
- Although rare to access items below the top few, there are so many that it yields a “heavy tailed” distribution
- Substantial value from even a tiny cache
- Substantial misses from even a very large cache

Demand paging cost model

- Since demand paging like caching, can compute average access time! (“Effective Access Time”)
 - $EAT = \text{Hit Rate} \times \text{Hit Time} + \text{Miss Rate} \times \text{Miss Time}$
 - $EAT = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$
- Example:
 - Memory access time = 200 nanoseconds
 - Average page-fault service time = 8 milliseconds
 - Suppose p = Probability of miss, $1-p$ = Probability of hit
 - Then, we can compute EAT as follows:
$$EAT = 200\text{ns} + p \times 8 \text{ ms} = 200\text{ns} + p \times 8,000,000\text{ns}$$
- If one access out of 1,000 causes a page fault, then $EAT = 8.2 \mu\text{s}$:
 - This is a slowdown by a factor of 40!
- What if want slowdown by less than 10%?
 - $EAT < 200\text{ns} \times 1.1 \Rightarrow p < 2.5 \times 10^{-6}$
 - This is about 1 page fault in 400,000!

What factors lead to misses in page cache?

- **Compulsory Misses:**

- Pages that have never been paged into memory before
- How might we remove these misses?
 - Prefetching: loading them into memory before needed
 - Need to predict future somehow! More later

- **Capacity Misses:**

- Not enough memory. Must somehow increase available memory size.
- Can we do this?
 - One option: Increase amount of DRAM (not quick fix!)
 - Another option: If multiple processes in memory: adjust percentage of memory allocated to each one!

What factors lead to misses in page cache?

- **Conflict Misses:**

- Technically, conflict misses don't exist in virtual memory, since it is "fully-associative"

- **Policy Misses:**

- Caused when pages were in memory, but kicked out prematurely because of replacement policy
- How to fix? Better replacement policy

Summary

- “Translation Lookaside Buffer” (TLB)
 - Small number of PTEs and optional process IDs (< 512)
 - Often Fully Associative (Since conflict misses expensive)
 - On TLB miss, page table must be traversed and if located PTE is invalid, cause Page Fault
 - On change in page table, TLB entries must be invalidated
- Demand Paging: Treating the DRAM as a cache on disk
 - Page table tracks which pages are in memory
 - Any attempt to access a page that is not in memory generates a page fault, which causes
- OS to bring missing page into memory

Thanks